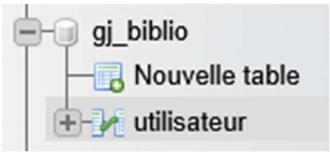
Lien avec une base de données

Pour une authentification plus sérieuse, les identifiants autorisés vont être stockées dans une base de données.

Création de la base de données

Sous PhpMyAdmin, créez une base xx_biblio avec vos initiales pour xx.
Puis ajoutez une table utilisateur, avec cinq champs : le login (clé primaire), le mot de passe, le nom, le prénom et le type de compte (A pour administrateur, L pour lecteur…).



#	Nom	Type
1	login	varchar(30)
2	password	varchar(100)
3	nom	varchar(50)
4	prenom	varchar(50)
5	compte	char(1)

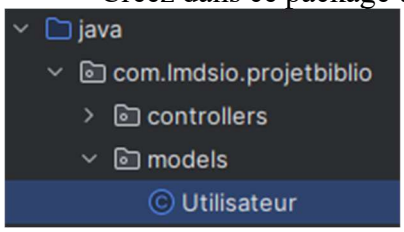
Ajoutez des utilisateurs, avec des mots de passe hashés (depuis le site https://bcrypt.online/ par exemple)

login	password	nom	prenom	compte
admin	$2y$10$uT.iKzCyMTaLA3/DJ0pe.OhHnjmigsHAOTz1bdqzACc...	Administrateur		A
jpalir	$2y$10$O6qY763XuqT.3xGE0cDPCOYnZXT5i3VMVhm4ddEBrRR...	Palir	José	L

Modèle Utilisateur

Ajoutons une classe métier Utilisateur pour manipuler les données correspondantes :

- Créez un nouveau package com.lmdsio.projetbiblio.models
- Créez dans ce package un nouveau fichier de classe Utilisateur



Pour chaque champ de la table utilisateur, codez une propriété dans la classe avec ses méthodes getter et setter :

```
public class Utilisateur {
    private String login, password;
    private String nom, prenom;
    private char compte;

    public String getLogin() {
        return login;
    }
    public void setLogin(String login) {
        this.login = login;
    }
    // A COMPLETER...
```

Ainsi qu'un constructeur avec le login :

```
    public Utilisateur(String login) {
        // A COMPLETER
    }
```

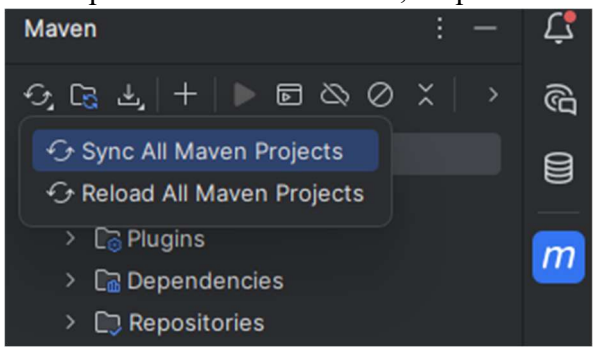
Ajout de la bibliothèque mysql-connector-java au projet

Cette bibliothèque va permettre l'accès à une base MySQL via l'API JDBC (Java Database Connectivity)
Dans le fichier **pom.xml**, ajoutez une dépendance pour inclure la version 8.0.33 au projet :

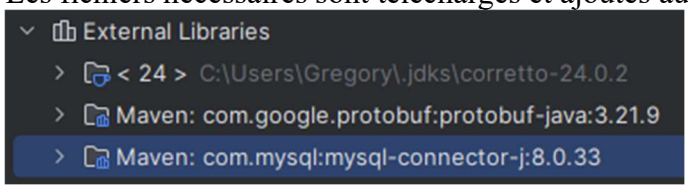
```
<dependencies>
    <dependency>
        <groupId>mysql</groupId>
        <artifactId>mysql-connector-java</artifactId>
        <version>8.0.33</version>
    </dependency>
```

Le texte est en rouge car les fichiers ne sont pas encore présents dans le projet.

Sur la partie droite de l'éditeur, cliquez sur l'icône *m* de Maven, puis sur « Sync Maven Projects » :



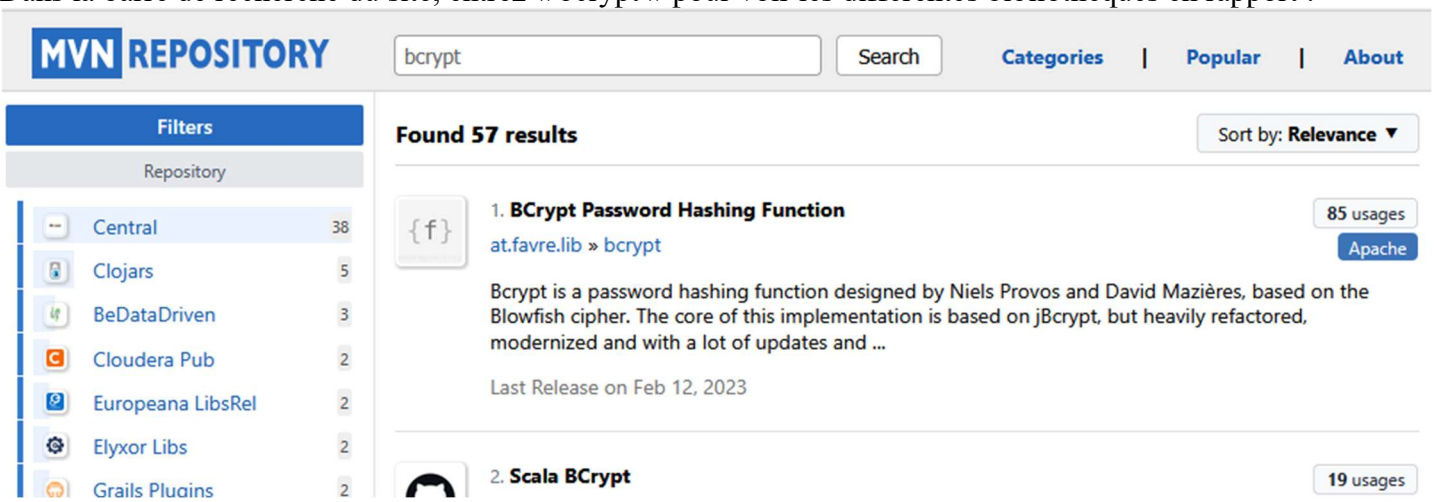
Les fichiers nécessaires sont téléchargés et ajoutés au projet, visibles dans le dossier External Libraries :



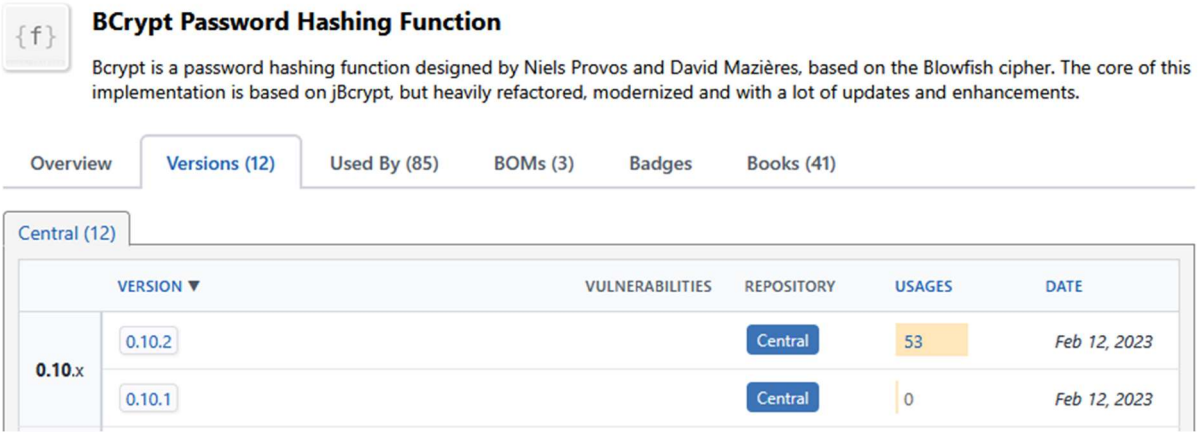
Ajout de la bibliothèque bcrypt au projet

Cette bibliothèque va permettre la vérification des mots de passe hashés.
Vous allez cette fois rechercher la dépendance depuis le site de dépôt Maven https://mvnrepository.com/

Dans la barre de recherche du site, entrez « bcrypt » pour voir les différentes bibliothèques en rapport :

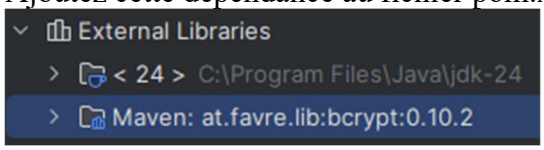


Cliquez sur la bibliothèque « Bcrypt Password Hashing Function », puis sur la dernière version (0.10.2 par ex)



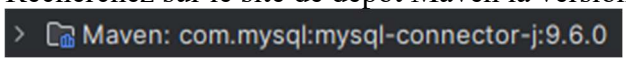
Vous trouvez alors dans la page correspondant à la version la dépendance nécessaire pour un projet Maven :

Ajoutez cette dépendance au fichier pom.xml, puis synchronisez le projet.



Mise à jour de la bibliothèque mysql-connector-java

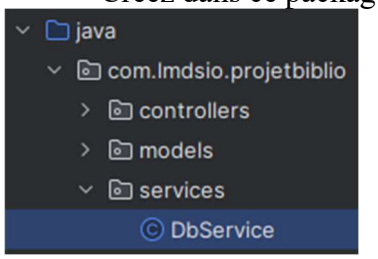
La bibliothèque **mysql-connector-java 8.0.33** contient des failles de sécurité (par ex CVE-2023-22102) ! Recherchez sur le site de dépôt Maven la version la plus récente et mettez à jour la dépendance dans le projet.



Service DbService

Pour bien séparer le traitement des données (dans les contrôleurs) et le lien avec la base de données (dans des services), nous allons mettre en place un service qui sera chargé d'accéder à la base :

- Créez un nouveau package com.lmdsio.projetbiblio.services
- Créez dans ce package un nouveau fichier de classe DbService



Cette classe DbService va gérer la connexion avec la base de données, grâce à l'objet Connection de java.sql.
Ajoutez pour cela le champ privé dbConn :

```
DbService.java
package com.lmdsio.projetbiblio.services;

import java.sql.*;

public class DbService {
    private Connection dbConn = null;
}
```

Ajoutez ensuite dans la classe deux fonctions pour se connecter ou se déconnecter de la base.
La fonction **dbConnect** permet d'initialiser une connexion :

```
protected void dbConnect() throws SQLException {
    String db = "jdbc:mysql://" + "localhost:3306/gj_biblio";
    String dbUser = "usersio";
    String dbPwd = "sio";
    dbConn = DriverManager.getConnection(db, dbUser, dbPwd);
}
```

La méthode getConnection de l'objet DriverManager de JDBC va se connecter à la base de données à partir des paramètres fournis : l'adresse du serveur de base de données (localhost sur le port 3306), le nom de la base (xx_biblio) et l'utilisateur de la base avec son mot de passe.
Si la connexion est correctement initialisée, elle est mémorisée dans le champ privé dbConn. Sinon une exception de type SQLException sera levée par la méthode getConnection.

La fonction **dbClose** refermera la connexion ouverte quand on n'en aura plus besoin :

```
protected void dbClose() throws SQLException {
    dbConn.close();
}
```

Ajoutez enfin une fonction **dbQuery** pour exécuter des requêtes de sélection :

```
protected ResultSet dbQuery(String sql, String param1, String param2) throws SQLException {
    PreparedStatement stmt = dbConn.prepareStatement(sql);
    if (param1 != null) stmt.setString(1, param1);
    if (param2 != null) stmt.setString(2, param2);
    return stmt.executeQuery();
}
```

Cette fonction prend en argument une requête SQL et deux paramètres facultatifs.
Elle exécute une requête préparée pour éviter les injections SQL (voir le cours de cybersécurité).
Les paramètres de la requête sont indiqués par des points d'interrogation.
Le résultat est retourné dans un objet ResultSet qui est une collection d'enregistrements.

Par exemple, on pourrait appeler la fonction de la manière suivante :

```
ResutSet rs = dbQuery("SELECT * from utilisateur WHERE login=?","admin",null);
```

Le point d'interrogation sera remplacé par 'admin'. Il n'y a pas de deuxième paramètre dans cette requête.
La requête SQL sécurisée sera : `SELECT * from utilisateur WHERE login='admin'`

<u>Remarque</u> : les fonctions sont déclarées protected, donc accessibles uniquement à des classes dérivées.

Service UtilisateurService

La classe UtilisateurService va dériver de DbService pour pouvoir utiliser la connexion à la base de données. Elle va permettre de gérer les requêtes en rapport avec la table utilisateur.

- Créez dans le package services un nouveau fichier de classe UtilisateurService
- Indiquez que cette classe dérive de DbService

```
UtilisateurService.java
1   package com.lmdsio.projetbiblio.services;
2
3   public class UtilisateurService extends DbService {
4   }
```

Ajoutez dans la classe une propriété **user** pour stocker l'utilisateur connecté, avec sa méthode get :

```
static private Utilisateur user = null;
public Utilisateur getUser() {
    return user;
}
```

Remarque : cette propriété est déclarée static, donc commune à toutes les instances de la classe.

Ajoutez ensuite la fonction **signIn** dans la classe :

```
public boolean signIn(String login, String password) throws Exception {
    boolean ok = false;
    try {
        dbConnect();
        ResultSet rs = dbQuery("SELECT * from utilisateur WHERE login =?", login, null);
        while (rs.next()) {
            String hash = rs.getString("password");
            if (BCrypt.verifyer().verify(password.toCharArray(), hash).verified) {
                user = new Utilisateur(rs.getString("login"));
                user.setNom(rs.getString("nom"));
                user.setPrenom(rs.getString("prenom"));
                user.setCompte(rs.getString("compte").charAt(0));
                ok = true;
            }
        }
        dbClose();
    } catch (SQLException e) {
        throw new Exception("Problème d'accès aux identifiants !");
    }
    return ok;
}
```

N'oubliez pas d'importer les classes nécessaires :

```
import com.lmdsio.projetbiblio.models.Utilisateur;
import java.sql.ResultSet;
import java.sql.SQLException;
import at.favre.lib.crypto.bcrypt.BCrypt;
```

Observez le code pour bien comprendre le fonctionnement :

La fonction **signIn** va vérifier l'authentification à partir des identifiants dans la table utilisateur.
La variable booléenne ok sera vraie uniquement si les identifiants sont acceptés :

```
boolean ok = false;
```

On se connecte à la base de données avec la méthode dbConnect héritée de DbService :

```
dbConnect();
```

On exécute alors la requête "SELECT * from utilisateur WHERE login=?" avec la variable login comme paramètre. On récupère ainsi les résultats dans un objet **ResultSet** rs :

```
ResultSet rs = dbQuery("SELECT * from utilisateur WHERE login =?", login, null);
```

La méthode **next** permet de parcourir un par un chaque résultat. Elle renvoie faux quand il n'y en a plus :

```
while (rs.next()) {
```

Dans la boucle de parcours, la méthode **getString** permet d'obtenir une chaîne de caractères à partir du nom d'une colonne de résultat. On récupère ainsi la colonne password de la base dans une variable hash :

```
String hash = rs.getString("password");
```

On utilise alors la méthode **verify** de la bibliothèque BCrypt pour comparer le hash et le mot de passe :

```
if (BCrypt.verifyer().verify(password.toCharArray(), hash).verified) {
```

Si cela correspond, alors on crée un objet Utilisateur référencé par la propriété user, que l'on initialise avec les champs issus de la base. On ne mémorise pas le mot de passe qui ne nous servirait à rien puisqu'il est hashé :

```
    user = new Utilisateur(rs.getString("login"));
    user.setNom(rs.getString("nom"));
    user.setPrenom(rs.getString("prenom"));
    user.setCompte(rs.getString("compte").charAt(0));
```

Et la variable ok devient vraie :

```
    ok = true;
```

Enfin, on peut terminer la connexion à la base :

```
dbClose();
```

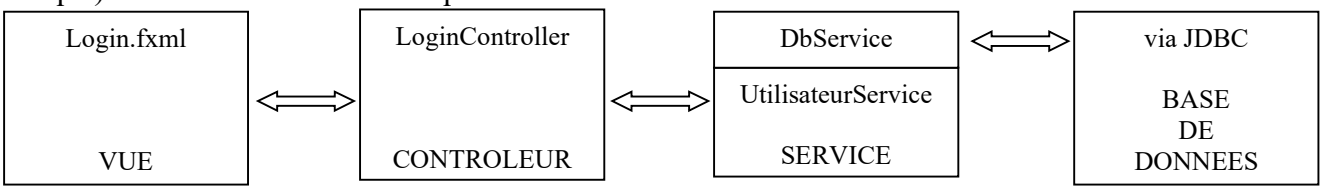
L'ensemble du processus de connexion à la base de données est englobé dans un bloc **try catch** afin d'intercepter toute erreur grâce à l'exception **SQLException**.
En cas d'erreur, on lève une nouvelle Exception générique avec un message explicatif :

```
try {...} catch (SQLException e) {
    throw new Exception("Problème d'accès aux identifiants !");
}
```

S'il n'y a pas d'erreur, alors on retourne finalement la valeur ok :

```
return ok;
```

Ainsi quand on utilisera depuis un contrôleur cette fonction d'authentification, le contrôleur saura si on a pu s'authentifier ou non (identifiants corrects ou incorrects), ou bien s'il y a eu un problème (exception générique). Mais seul le service sait qu'il a affaire à une base de données.



Codez enfin la fonction **signOut** qui efface simplement l'utilisateur connecté en réinitialisant la propriété user :

```
public boolean signOut() {
    user = null;
    return true;
}
```

Utilisation du service par le contrôleur

Nous allons maintenant modifier le code de connexion dans le contrôleur LoginController, pour utiliser le service UtilisateurService :

```
import com.lmdsio.projetbiblio.services.UtilisateurService;
```

```
@FXML
private void onLoginClick() {
    String login = loginField.getText();
    String password = passwordField.getText();

    UtilisateurService service=new UtilisateurService();
    try {
        if (service.signIn(login, password)) {
            loginText.setText("Hello " + service.getUser().getLogin() + " ! ");
            loginText.setStyle("-fx-text-fill: green");
        } else {
            loginText.setText("Identifiants incorrects !");
            loginText.setStyle("-fx-text-fill: red");
        }
    }
    catch (Exception e) {
        loginText.setText(e.getMessage());
        loginText.setStyle("-fx-text-fill: orange");
    }
}
```

On instancie un service UtilisateurService et l'on stocke une référence dans la variable service :

```
UtilisateurService service=new UtilisateurService();
```

On peut alors appeler la fonction signIn du service pour demander une connexion.
Si elle renvoie true, on affiche en vert le login de l'utilisateur connecté :

```
if (service.signIn(login, password)) {
    loginText.setText("Hello " + service.getUser().getLogin() + " ! ");
    loginText.setStyle("-fx-text-fill: green");
```

Sinon on prévient que les identifiants ont été refusés en rouge :

```
} else {
    loginText.setText("Identifiants incorrects !");
    loginText.setStyle("-fx-text-fill: red");
}
```

L'appel au service est englobé dans un bloc try catch. Cela permet d'intercepter toute erreur et d'afficher le message correspondant en orange :

```
try {...}
catch (Exception e) {
    loginText.setText(e.getMessage());
    loginText.setStyle("-fx-text-fill: orange");
}
```

Testez les trois cas de figure.

Hello admin ! Identifiants incorrects ! Problème d'accès aux identifiants !

Exercice d'application

Utilisateur connecté

- Ajoutez une entrée Bienvenue au menu Utilisateur, qui affichera la vue welcome.
- Créez un contrôleur WelcomeController et indiquez dans la page welcome le login, le nom et le prénom de l'utilisateur connecté, s'il y en a un.

Déconnexion

- Dans le contrôleur MasterController, utilisez la fonction signOut du service UtilisateurService pour déconnecter l'utilisateur courant lorsque l'on clique sur le menu Déconnexion.
- Si la déconnexion réussit, affichez la vue welcome.

Connexion

De même, lorsque la connexion réussit, on voudrait revenir directement à la vue welcome.
Mais le contrôleur LoginController n'a aucun accès à la fonction showContent qui se trouve dans le contrôleur principal MasterController.

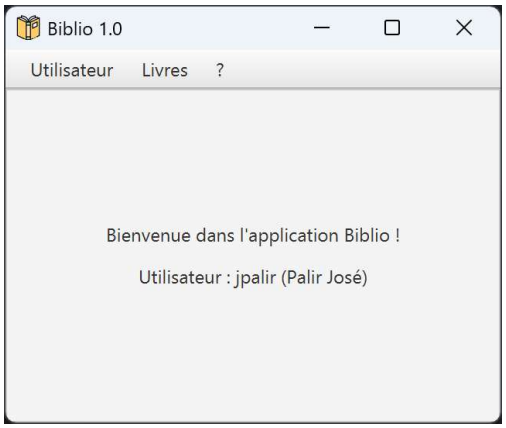
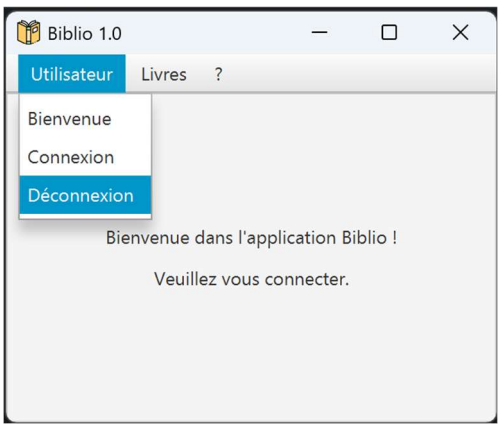
Une solution est de stocker une référence au MasterController accessible à chaque contrôleur de contenu :

- Créez une classe ContentController, avec une propriété static master et un setter static :

```
public class ContentController {
    static protected MasterController master;
    static public void setMaster(MasterController mc) { /* A COMPLETER */ }
}
```

- Initialisez la propriété master dans la fonction initialize du MasterController :

```
ContentController.setMaster(this);
```

- Changez la visibilité de la fonction showContent pour la rendre publique.
- Indiquez que la classe LoginController dérive de ContentController.
  Ainsi elle aura accès à la propriété master, et à ses fonctions publiques.
- Enfin, lors d'une connexion réussie, vous pouvez afficher directement la vue welcome.

⇨ **Déposez un fichier zip de votre projet**